

SPSC_Head_to_Head

SPSC Head to Head Benchmark

Github: [MarketDataPipeline](#)

The goal of this series of testing was to compare my SPSC queue implementation against other implementations used across companies today. The following SPSC queues were used:

- `rigtorp`: <https://github.com/rigtorp/SPSCQueue>
- `folly`: <https://github.com/facebook/folly>
- `moodycamel`: <https://github.com/ameron314/readerwriterqueue>
- `Boost`: <https://github.com/boostorg/lockfree>

Hardware Specification

	Dell Precision 3640 (x86-64)
CPU	Intel i9-10900, 10C/20T, Comet Lake
Architecture	x86-64
OS	Ubuntu 24.04 (bare metal)
RAM	32 GB
Kernel	6.8.0-28-generic
Cache Line	64 bytes
L1d / L1i	32 KiB / 32 KiB per core
L2	256 KiB per core
L3	20 MiB (shared)
Core Isolation	<code>isolcpus=0,1,2,3</code> + <code>nohz_full</code> + <code>rcu_nocbs</code>

Implementation Notes

My current implementation of the SPSC queue utilizes the following optimizations:

- Buffer elements are each aligned to 64 bytes, such that each element utilizes its own cacheline
- A thread-local variable is kept for tracking. This prevents needing to pull the true index variables.
 - Our cached head is stored on the same cache line as our tail index. This is because both are owned by the producer thread.
 - Our cached tail is stored on the same cache line as our head index. This is because both are owned by the consumer thread.
 - Instead of needing to do `head/tail.load(acquire)` every single time, which would force a cross-thread cache line transfer, we use cached indexes as a stale guess. Only when the

cached value is full/empty do we retrieve the true value via `acquire`.

- Example: This works because tail will be incrementing, and at some point (circular buffer) will wrap back around, while our cached head never changed. Once we see we are "full" based on a stale cached index, we update the cached index to the true value and recheck for our full condition.
- Our atomic indexes are aligned to 64 bytes such that they utilize their own cache line.

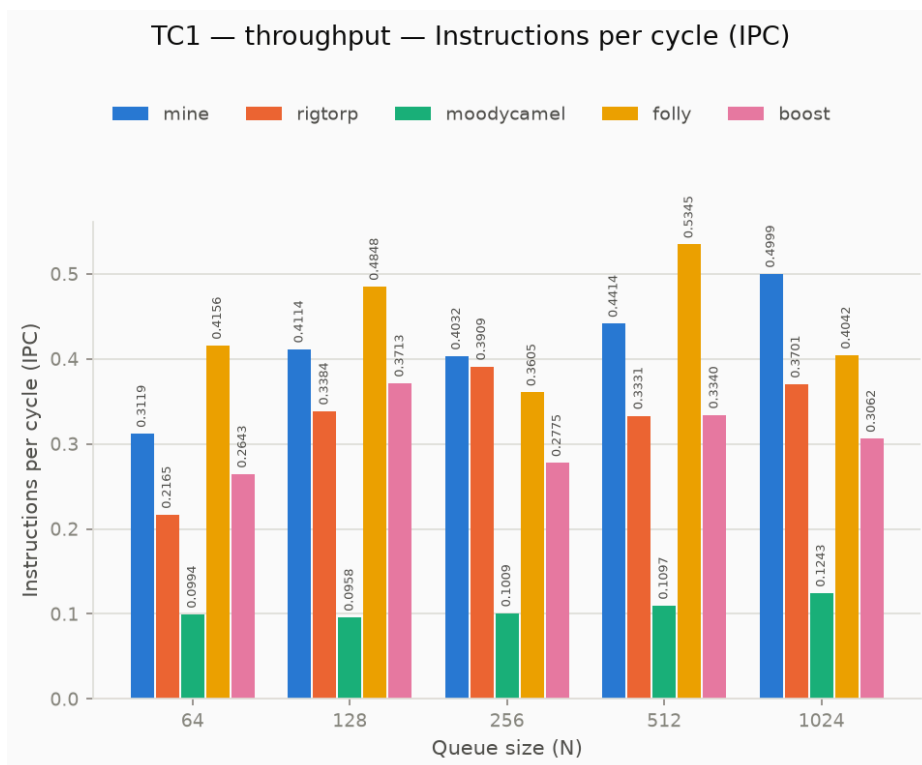
Testing

All testing was recorded using the `perf` tool. Using perf we grabbed cycles, instructions, LLC-load-misses, L1-dcache-load-misses, and the time to run. In the GitHub repo you can find the script used for collecting data here: `scripts/spsc_queue_head_to_head/spsc_head_to_head.sh`. A python script is also called to dynamically graph and view data.

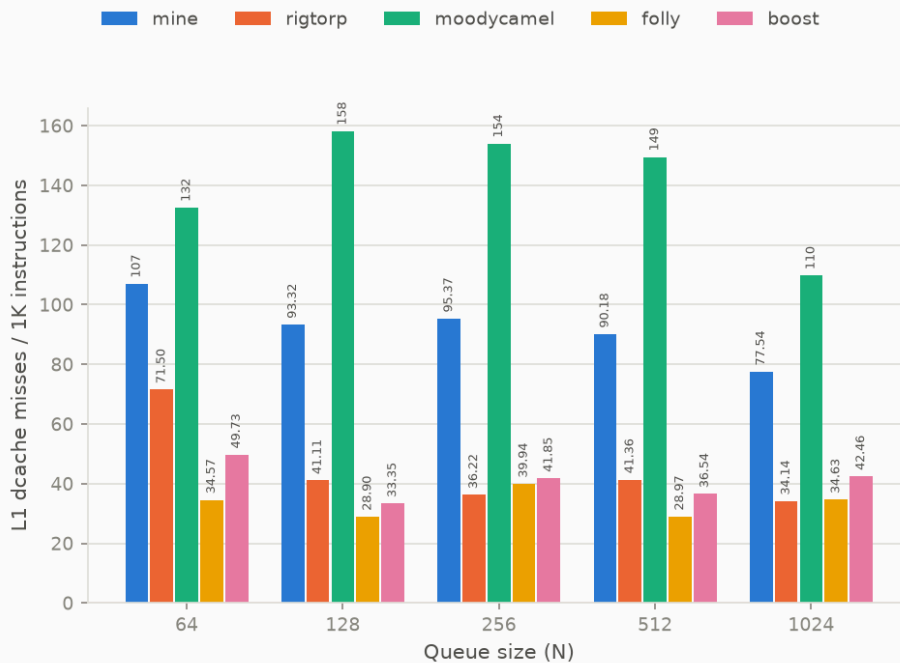
Each target, test case, and queue size (64, 128, 256, 512, 1024) are tested 3 times, utilizing a median for our recorded output.

Throughput

This test is designed to rapidly push and pop items to/from the queue. This test utilized 10,000,000 iterations.



TC1 — throughput — L1 dcache misses / 1K instructions



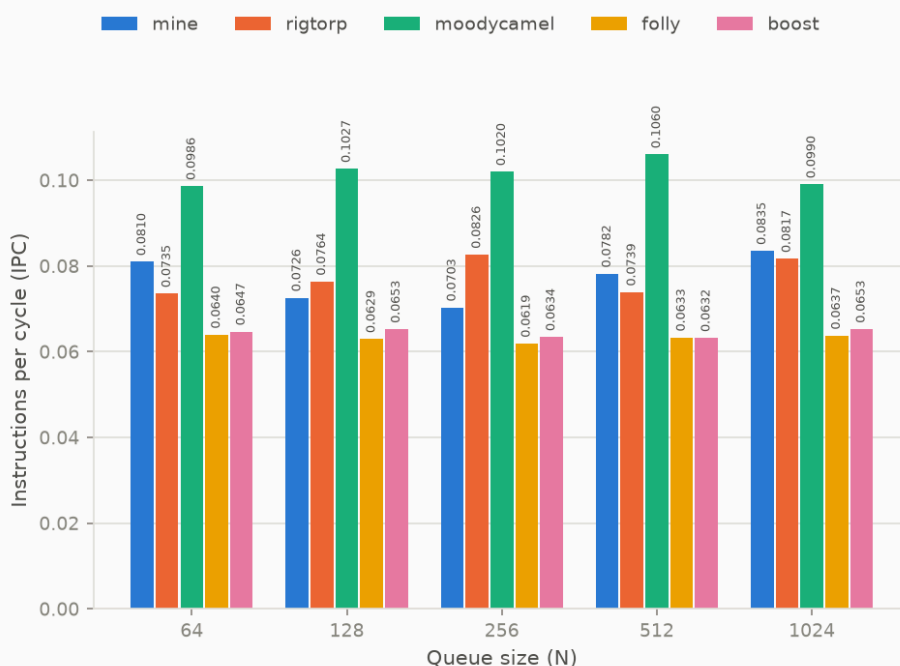
Request/Respond

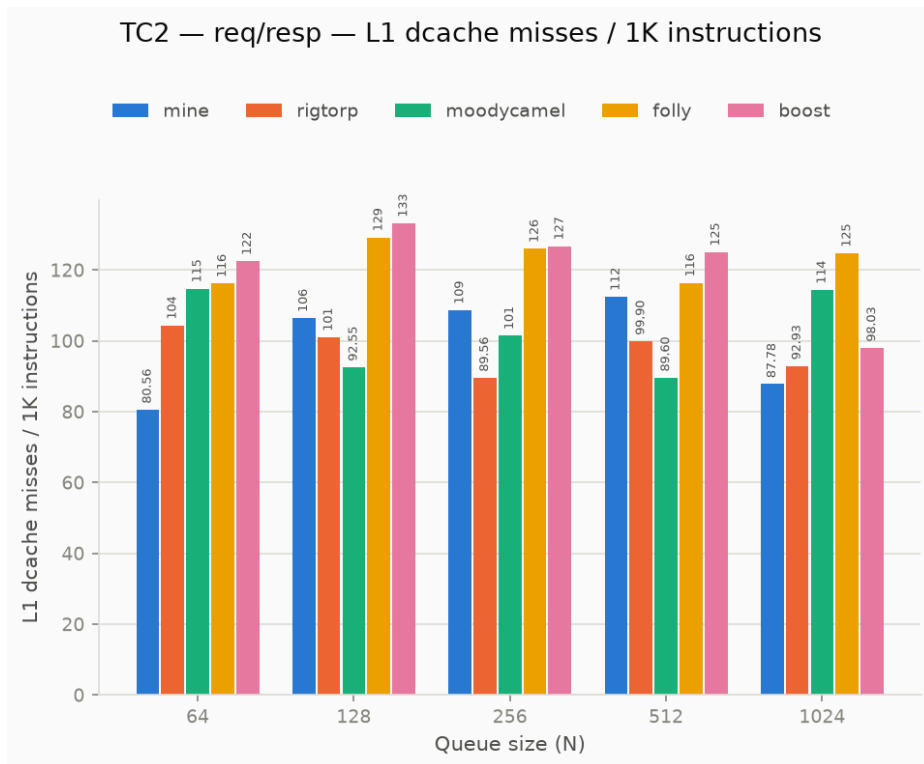
This test is designed to have 2 SPSC queues. One SPSC queue is a requester, and the other is a responder. As such, the order of operations is:

1. Thread 1: Queue 0 pushes X.
2. Thread 2: Queue 0 Pops X
3. Thread 2: Queue 1 Pushes X
4. Thread 1: Queue 1 Pops X

As you can see, we have a back and forth going on between two threads. The test also utilized 10,000,000 iterations.

TC2 — req/resp — Instructions per cycle (IPC)





Burst

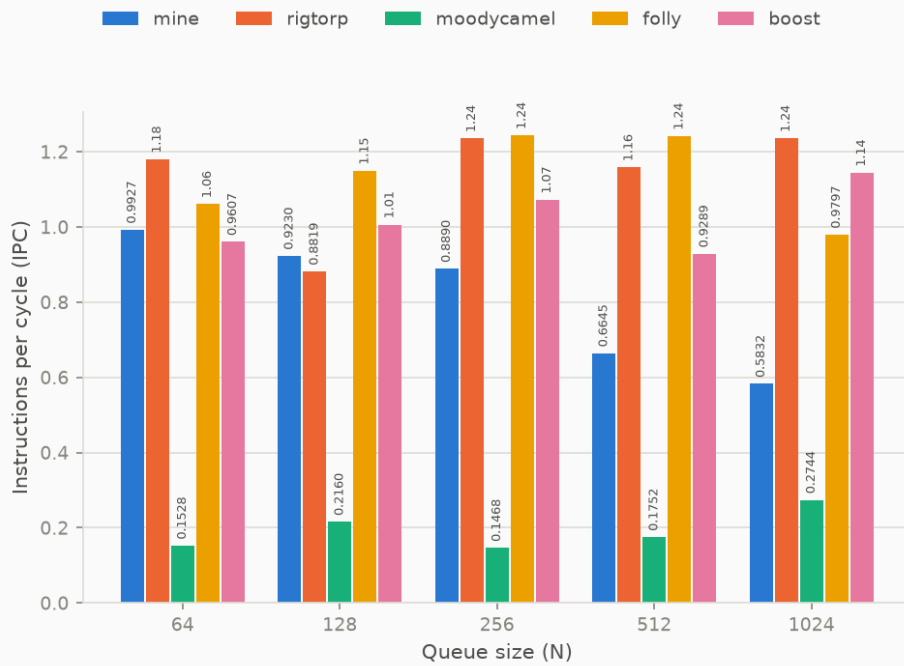
This test is to mimic a more real world situation. To do this we push 64 items to the queue, then we yield the producer thread. While the producer thread does nothing, our consumer pops all the data out. We do this for 100000 batches.

We also utilized percentiles here, as we tested across different queue sizes. We will only visualize 64, 512, and 1024. Our reasoning here is due to cache size. The L1 cache on the hardware the binary was tested on is 32KB. As such, 512 items at 64 bytes is 32KB. At 1024 our buffer is larger than the L1 cache, and so we must utilize the L2 cache.

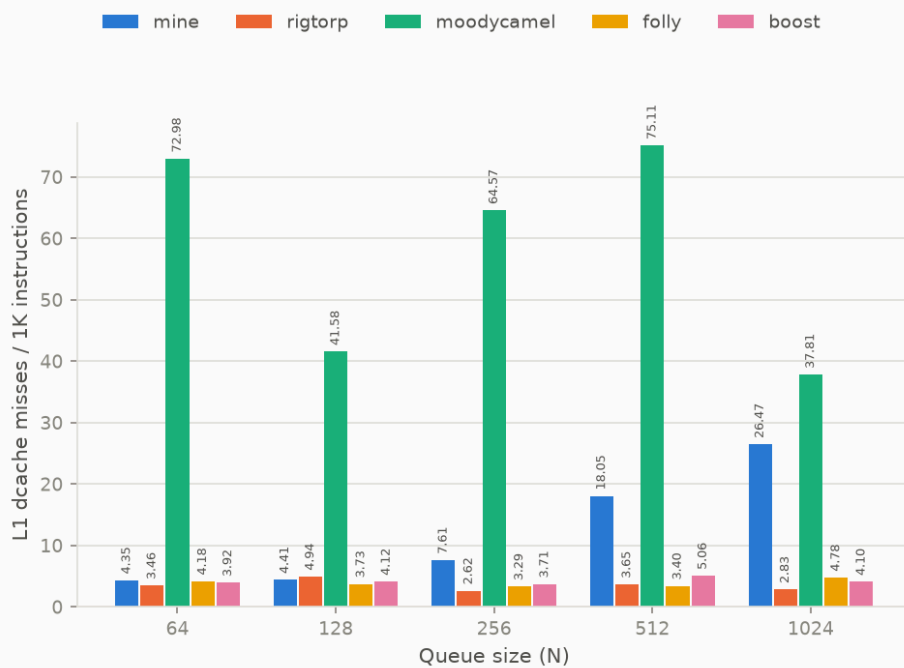
Note:

Some SPSC queues have a "Batch" function. We did not utilize these, as my implementation does not create its own batch.

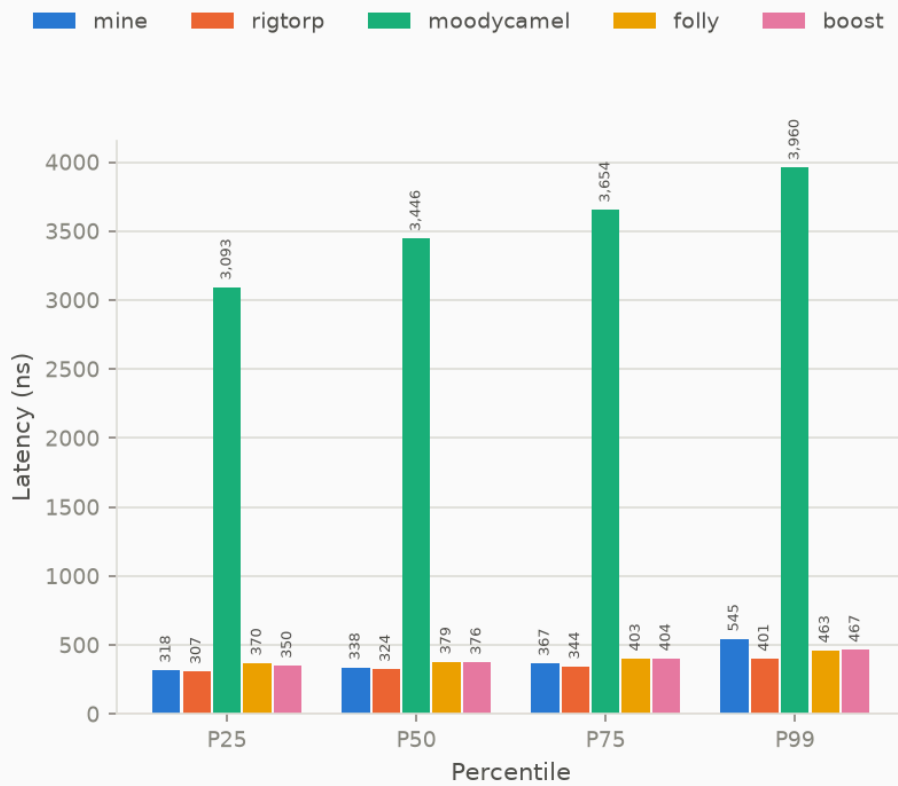
TC3 — burst drain — Instructions per cycle (IPC)



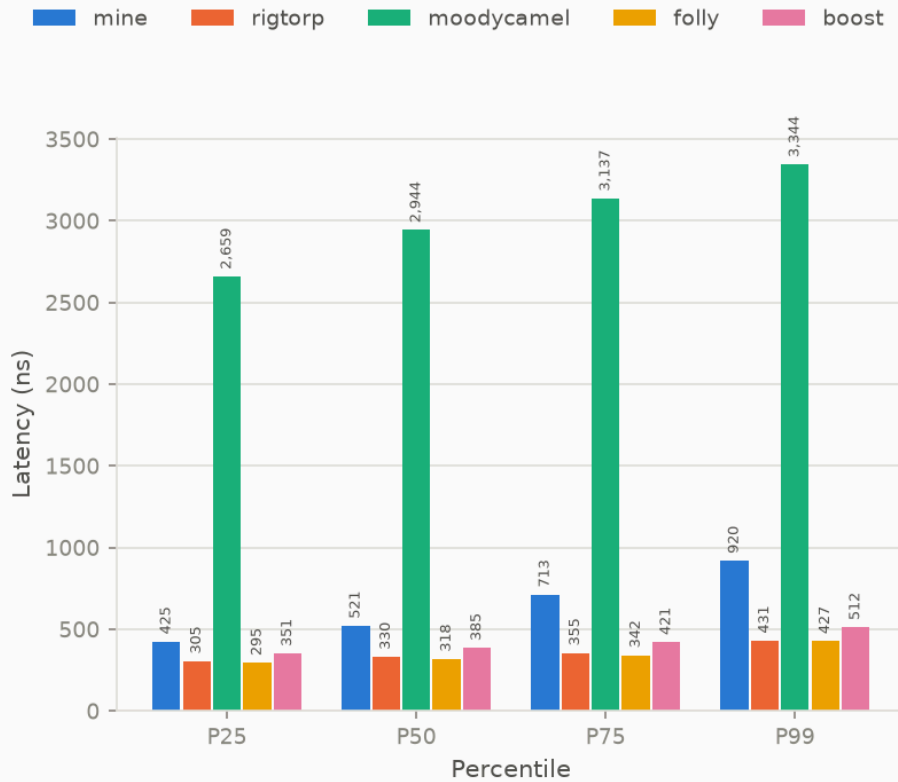
TC3 — burst drain — L1 dcache misses / 1K instructions



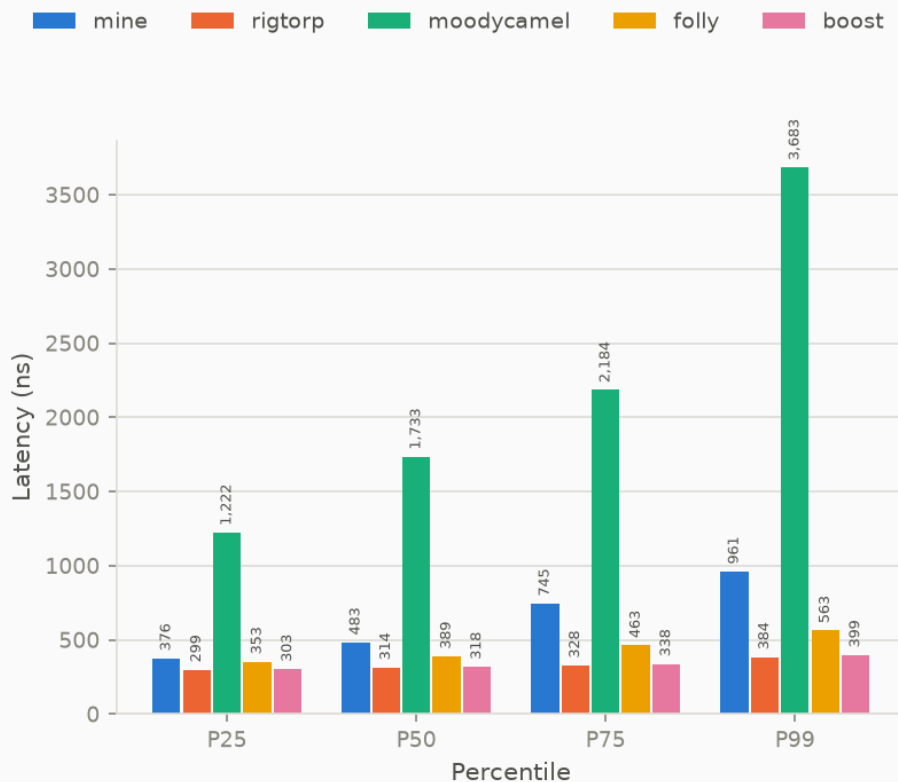
TC3 burst latency percentiles — N=64



TC3 burst latency percentiles — N=512



TC3 burst latency percentiles — N=1024



Key Observations

- `MoodyCamel::ReaderWriterQueue` is consistently slower across all three Test Cases, ranging from ~2x in Request/Respond to ~9x in Burst. `ReaderWriterQueue` is designed for variable-size messages and carries per-operation overhead that the fixed-size queues avoid.
- My implementation leads TC1 at larger queue sizes, achieving the highest IPC at N=1024 (0.500) vs `folly` (0.404), `rigtorp` (0.370), and `boost` (0.306). My IPC grows most consistently with queue size (0.312 at N=64 → 0.500 at N=1024). The cached-index optimization reduces total instruction count by ~16% vs `rigtorp` at N=1024 (298M vs 362M instructions), keeping the pipeline busier with less spinning.
- The cached-index optimization trades instruction count for L1 miss rate. In TC1 at N=1024, mine incurs 77.5 L1 misses per thousand instructions vs `rigtorp`'s 34.1 and `folly`'s 34.6. Fewer total loads are issued, but each cross-thread load is more likely to miss L1 when it does occur. At large queue sizes the instruction reduction wins out.
- In TC3 burst, the higher L1 miss rate becomes a liability. My L1 misses climb to 18.0 per kinstr at N=512 vs `rigtorp`'s 3.65 and `folly`'s 3.40. The fill-then-drain pattern forces cross-cache-line transfers on every batch. P50 at N=512: `Folly` 318ns, `rigtorp` 330ns, `Boost` 385ns, Mine 521ns. Mine's tail is also wider: P99 920ns (1.8x its own median) vs `Folly` P99 427ns and `Boost` P99 512ns.
- LLC misses are uniformly low across all implementations. L1 is the differentiating cache level — the data stays in L2/L3, not DRAM. The performance gaps are driven by L1 miss rate, not cache pressure.